

Systemsoftware Übung 1

Sebastian Ohs 1476427

Aufgabe 1

Ein Programm ist eine Datei mit Code, der ausgeführt werden kann. Ein Prozess ist ein laufendes Programm mit eigenem Adressraum im Arbeitsspeicher, Ressourcen und mindestens einem Thread. Dabei ist wichtig, dass mehrere Prozesse isoliert voneinander ausgeführt werden, also keinen Zugriff auf die Ressourcen eines anderen Prozesses besitzen. Somit müssen Prozesse, um zusammenarbeiten zu können, miteinander kommunizieren, bspw. Über Datenpakete in Netzwerken. Threads in einem Programmcode dienen der Parallelisierung innerhalb einer Prozesses, also der gleichzeitigen Ausführung von Code mit mehreren Prozessorkernen. Threads greifen dabei auf einen gemeinsamen Speicher zu, was zu Data Races und entsprechend zu Fehlern in der Ausgabe führen kann.

Aufgabe 2

Amdahlsches Gesetz: $S(n,p) = \frac{1}{(1-p) + \frac{p}{n}}$

p = parallelisierbarer Teil, n = Anzahl der Threads

$p = 0,25, n = 1: S(1, 0,25) = \frac{1}{(1-0,25) + \frac{0,25}{1}} = 1$

$n = 2: S(2, 0,25) = \frac{1}{(1-0,25) + \frac{0,25}{2}} \approx 1,143$

$n = 4: S(4, 0,25) = \frac{1}{(1-0,25) + \frac{0,25}{4}} \approx 1,231$

$n = 8: S(8, 0,25) = \frac{1}{(1-0,25) + \frac{0,25}{8}} \approx 1,28$

$p = 0,5, n = 1: S(1, 0,5) = \frac{1}{(1-0,5) + \frac{0,5}{1}} = 1$

$n = 2: S(2, 0,5) = \frac{1}{(1-0,5) + \frac{0,5}{2}} \approx 1,33$

$n = 4: S(4, 0,5) = \frac{1}{(1-0,5) + \frac{0,5}{4}} = 1,6$

$n = 8: S(8, 0,5) = \frac{1}{(1-0,5) + \frac{0,5}{8}} \approx 1,778$

$p = 0,75, n = 1: S(1, 0,75) = \frac{1}{(1-0,75) + \frac{0,75}{1}} = 1$

$n = 2: S(2, 0,75) = \frac{1}{(1-0,75) + \frac{0,75}{2}} = 1,6$

$n = 4: S(4, 0,75) = \frac{1}{(1-0,75) + \frac{0,75}{4}} \approx 2,286$

$n = 8: S(8, 0,75) = \frac{1}{(1-0,75) + \frac{0,75}{8}} \approx 2,909$

Aufgabe 3

Ich habe die parallele Variante des Codes, wie in der Vorlesung auf die Dateien des aktuellen Linux-Kernels angewandt. Explizit habe ich die Zeilen der C-Programme zählen lassen. Bei einem Thread benötigte der Code insgesamt 14565 ms. Bei zwei Threads verringerte sich die Dauer stark auf 2868 ms. Bei vier Threads verringerte sich die Dauer nicht mehr, sondern blieb mit 2989 ms ähnlich der Zeit mit zwei Threads. Auch bei acht Threads mit 2748 ms und 16 Threads mit 3182 ms lässt sich eher eine Stagnation der Ausführungsgeschwindigkeit beobachten. Daraus lässt sich schließen, dass entweder der nicht parallelisierbare Teil meines Codes sehr hoch ist, oder aber die Verteilung der zu überprüfenden Dateien an die Threads nicht ausbalanciert ist.